Experimental Methods in Computer Science

Experiment Design

André Miguel Correia Cardoso 2022222265

Luis Miguel Ferreira Vaz 2022214707

Pedro Costa 2022220304

University of Coimbra, Mestrado em Engenharia Informática 2025/2026

Contents

1 Introduction **3**

2 Experimental Design **3**

2.1 3

3 Bug Finding **3**

3.1 Experimental Design 4

3.1.1 Research Question 4

3.1.2 Independent Variables 4

3.1.3 Dependent Variables 4

3.1.4 Controlled Variables 4

4 Code Generation **5**

4.1 Context and Procedure Description 5

4.2 Variables, Research Questions and Hypotheses 6

4.3 Variables 6

4.3.1 Independent Variables 6

4.3.2 Dependent Variables 6

1 Introduction

Large Language Models (LLMs) are increasingly being integrated into software development workflow. As their use becomes more widespread, it is important to evaluate the extent to which developers can rely on these tools in practical software engineering contexts.

This report was developed as part of the Experimental Methods in Computer Science curricular unit and addresses the following central problem statement:

Can developers rely on LLM-based tools to support software development activities?

Given the wide scope of this question, the experiment focuses on two specific areas of software development: Code Generation and Code Review (more specifically Bug Finding). These two areas were studied separately, using distinct datasets and statistical analysis methods suited to each task.

Despite these differences, both parts of the experiment share a common setting: they are based on the Python programming language and evaluate the same LLM models. This allows for a structured comparison of model performance across two relevant software development activities, contributing to a broader understanding of the capabilities and limitations of LLM-based tools.

2 Experimental Design

2.1

3 Bug Finding

Code Review, and more specifically Bug Finding, refers to the practice of providing an LLM with source code and asking it to inspect the code for potential defects. Depending on the experimental setup, this process may involve supplying only the relevant code files, or it may include additional contextual information such as failing tests, execution outputs, error messages, or brief descriptions of the expected behaviour.

This experiment focuses on the bug detection capabilities of the selected LLMs. In particular, the models are tasked with analysing Python source code and identifying the locations of real bugs. The objective is to evaluate how effectively each model can support developers in detecting defects during code review activities.

3.1 Experimental Design

3.1.1 Research Question

The research question for this experiment is:
How does the choice of Large Language Model affect bug-finding performance in real-world Python bugs?

3.1.2 Independent Variables

The independent variables are:

- **LLM Model (2 levels)**:
- **Language (1 level)** :
  - Python

  Even though there is only one language bing used in the bug, it is still considered an independent variable.

3.1.3 Dependent Variables

The dependent variables used in the statistical comparison are:

- **Recall**: Fraction of the real bugs identified.

$$Recall = \frac{TP}{TP + FN}$$

- **F1-Score**: F1-score combines recall and precision into a single metric that penalises both missed bugs and incorrectly reported bugs:

$$F1 = \frac{2TP}{2TP + FP + FN}$$

Precision was excluded, since, due to its formula, when an LLM reports zero bugs, it will generate a division by 0. Since F1-score is the harmonic mean between Recall and Precision we decided to opt for that dependent variable.

3.1.4 Controlled Variables

In order to reduce unnecessary variables and isolate each LLM's performance, we kept the following variable constant:

- **Scenarios**: Both LLMs are fed the same bug scenarios

- **Prompt**: All prompts follow the same format. They just vary from scenario to scenario in the Source Files section, since they include that specific scenario's buggy code.
- **New chat for each submission**: In order to avoid outputs based on previous context, each submission is made in a new chat.

4 Code Generation

4.1 Context and Procedure Description

This experiment evaluated the reliability of Large Language Models (LLMs) in generating SQL queries. SQL was selected because SQL programming problems provide a standardized evaluation environment in which generated solutions can be automatically executed and validated against predefined test cases.

A dataset consisting of 30 SQL programming problems was collected from the LeetCode platform. The selected problems were evenly distributed across three difficulty levels (Easy, Medium, and Hard) to enable the assessment of model performance under varying degrees of complexity.

For each problem, both LLMs received the same prompt template and were instructed to generate a single SQL query that solved the task. The generated solutions were subsequently submitted to the LeetCode evaluation platform, which automatically executed each query against a set of hidden test cases and determined whether the solution was accepted or rejected.

The evaluation was based on the acceptance status returned by the platform rather than a direct comparison of query syntax. This approach focuses on functional correctness, ensuring that semantically equivalent queries producing identical results are treated as valid solutions.

In addition to correctness, the execution time reported by the platform was recorded as a secondary metric. This metric is used as a proxy for query efficiency, allowing a comparative analysis of whether the generated solutions differ in computational performance between models. Although execution time is influenced by factors such as query structure and database optimization strategies, it provides a practical indicator of whether one model tends to generate more efficient (i.e., potentially more optimized) SQL solutions than the other.

Query correctness remains the primary measure of reliability, while execution time is treated as an auxiliary performance metric to support a secondary analysis of solution efficiency.

Each model was allowed a single attempt per problem. This decision was made to evaluate first-response reliability and to avoid the introduction of bias associated with iterative prompt refinement or multiple generation attempts.

To ensure experimental consistency, all problems were presented using the same prompt structure, with only the problem statement and associated input/output specifications varying between tasks. Furthermore, each problem was submitted in an independent conversation context, preventing information from previous interactions from influencing subsequent responses. As a result, every

generated solution constituted an independent observation.

4.2 Variables, Research Questions and Hypotheses

Is there a statistically significant difference in SQL query correctness between ChatGPT Mini and Gemini Flash?

Does problem difficulty affect the reliability of SQL solutions generated by LLMs?

4.3 Variables

The experimental design includes two independent variables and multiple dependent variables used to evaluate the reliability and performance of Large Language Models (LLMs) in SQL code generation.

4.3.1 Independent Variables

IV1: LLM Model

This variable represents the Large Language Model used to generate SQL queries. It has two levels:

- ChatGPT Mini
- Gemini Flash

IV2: Problem Difficulty

This variable represents the difficulty level of the SQL programming problems selected from the dataset. It has three levels:

- Easy
- Medium
- Hard

4.3.2 Dependent Variables

DV1: Query Outcome Classification (Primary Outcome)

This variable classifies the outcome of each generated SQL query based on its execution and correctness. It is defined as a categorical variable with the following levels:

- **Correct**: the query executes successfully and passes all hidden test cases

- **Incorrect Logic**: the query executes successfully but fails one or more hidden test cases due to logical or constraint violations
- **Syntax or Execution Error**: the query fails to execute due to syntax errors or runtime issues

This classification allows a more granular analysis of failure types beyond binary correctness.

DV2: Execution Success

This variable captures whether the generated SQL query is syntactically valid and executable. It is defined as:

- Executes successfully
- Execution error

DV3: Execution Time (Secondary Performance Metric)

This variable records the execution time reported by the evaluation platform for each query. It is treated as a continuous variable and used as a proxy for query efficiency. It is not used as a measure of correctness, but instead as a secondary performance indicator to compare computational efficiency across models.